Behavioral Cloning and Diffusion Policies for Robotic Stacking: A Comprehensive Benchmark on ManiSkill3 StackCube-v1

Wenqi Liao, Xinhui Chen, Xiaotong Wang, Hanwen Ye, Gaodi Yu
Faculty of Innovation Engineering, Macau University of Science and Technology

May 2026

Abstract

We present a comprehensive benchmark of imitation learning methods for the StackCube-v1 task in ManiSkill3, comparing behavioral cloning, diffusion policies, equivariant architectures, temporal models, Flow Matching, Deep Koopman operators, and classical PID control. Using 40,697 expert demonstration steps from a PPO-trained policy, we evaluate 18 distinct approaches. Our best diffusion model achieves an 84% success rate (100-episode eval), closely followed by a PID controller at 82%. We find that (1) increasing diffusion model capacity (hidden=384, depth=6) with cosine scheduling, EMA, SWA yields the best learned policy; (2) all attempts at rotation-equivariant or invariant architectures underperform; (3) temporal modeling and entity-aware attention provide no benefit; and (4) Flow Matching completely fails on this task. Our results suggest that for strongly-structured robotic stacking tasks, classical control remains competitive, and that the task's inherent structure—a fixed-base serial manipulator—limits the utility of geometric equivariance.

1 Introduction

Robotic manipulation has seen remarkable progress in recent years, driven by advances in both reinforcement learning (RL) and imitation learning (IL). Among IL approaches, behavioral cloning (BC)—directly regressing actions from observations using supervised learning—remains a widely-used baseline due to its simplicity and stability [1]. However, BC suffers from fundamental limitations: the mean-squared error (MSE) loss averages over multi-modal action distributions, and compounding errors accumulate during test-time rollouts [2].

Diffusion policies [3] address these issues by reformulating action generation as a conditional denoising process, naturally handling multi-modal distributions without mode averaging. Since their introduction, diffusion-based policies have achieved state-of-the-art results across diverse manipulation benchmarks.

In this work, we present a thorough empirical investigation of 18 different methods on the StackCube-v1 task from ManiSkill3 [8]. StackCube-v1 requires a Franka Emika Panda robot to pick a red cube and stack it atop a green cube on a tabletop—a contact-rich task demanding precise alignment and grasp execution. Our contributions are:

- A comprehensive benchmark of 9 diffusion policy variants (v1–v3, temporal, Koopman-boosted, CQE, entity-aware) alongside BC, BC-RNN, Flow Matching, and PID control.
- Investigation of rotation-equivariant/invariant backbones (C4/C8, SE(2) steerable, harmonic) applied to diffusion conditioning networks.
- Quantitative and qualitative analysis of 40,697 expert demonstration steps converted from a PPO policy.
- Key findings on the limits of equivariance for fixed-base manipulation and the surprising effectiveness of classical control.

2 Related Work

2.1 Imitation Learning for Manipulation

Behavioral cloning [1] is the most direct IL approach. For robotic manipulation, DAgger [2] addresses distribution shift via online data aggregation. More recently, implicit behavioral cloning [13] uses energy-based models to avoid mode averaging.

2.2 Diffusion Policies

Denoising diffusion probabilistic models (DDPM) [4] have become the dominant generative modeling paradigm. Song et al. [5] proposed Denoising Diffusion Implicit Models (DDIM) for accelerated deterministic sampling. Chi et al. [3] introduced Diffusion Policy, applying conditional diffusion to robot action generation. Our work builds directly on this framework, implementing noise-prediction networks with FiLM conditioning and cosine beta schedules [6].

2.3 Equivariant Architectures

Leveraging symmetry in robotic learning has been a recurring theme. Cohen and Welling [11] introduced steerable CNNs for SE(2) equivariance. Wang et al. [12] proposed Equivariant Diffusion Policy for manipulation. However, these methods assume symmetries in the observation space that may not hold for fixed-base manipulators—a key finding we confirm empirically.

2.4 Koopman Operator Theory

Koopman theory [14] posits that nonlinear dynamics can be linearly represented in an infinite-dimensional observable space. Huang et al. [9] recently proposed the Deep Koopman-boosted Dual-branch Diffusion Policy (D3P), which learns a latent linear dynamics model via a Koopman operator to improve robustness to out-of-distribution states. We implement and evaluate a state-based adaptation of this approach.

2.5 Flow Matching

Lipman et al. [7] proposed Flow Matching as a simulation-free alternative to continuous normalizing flows. Unlike diffusion models that learn a noise-to-data path, Flow Matching learns a straight velocity field. We evaluate an adaptation of this framework for StackCube.

2.6 Simulation Platform

All experiments use ManiSkill3 [8], a GPU-parallelized robotics simulator built on SAPIEN [10]. The StackCube-v1 task requires precise pick-and-stack manipulation.

3 Methods

3.1 Task and Data

StackCube-v1 provides a Franka Emika Panda robot with 7-DOF arm and a parallel-jaw gripper (8 action dimensions: 7 joint velocities + 1 gripper command). The observation space is 48-dimensional, comprising joint positions (qpos[0:8]), joint velocities (qvel[8:16]), previous action (prev_action[16:18]), TCP position (18:21) and quaternion (21:25), positions and quaternions of two cubes (25:39), goal position/quaternion (39:46), and extra features (46:48).

Expert demonstrations are drawn from a pre-trained PPO policy using the `pd_joint_delta_pos` controller, totaling 932 trajectories with 40,697 steps. The dataset is randomly split 90%/10% into training (36,627 steps) and validation (4,070 steps). All observations and actions are normalized to zero mean and unit variance using training set statistics.

3.2 Baseline Methods

BC-MLP (No. 1). A 3-layer MLP (256 neurons each, LayerNorm + GELU + Dropout 0.1) directly regressing actions from observations with MSE loss.

Diffusion v1 (No. 2). A basic DDPM policy with depth-4, hidden-256 NoisePredictor network using FiLM conditioning and linear beta schedule (T=100). Details follow Chi et al. [3].

Diffusion v3 (No. 3). The best-performing learned policy, scaling network capacity to hidden=384, depth=6 layers with cosine beta schedule [6], Exponential Moving Average (EMA, decay=0.999), action noise augmentation (std=0.01), and Stochastic Weight Averaging (SWA) over the last 50 epochs. Inference uses DDIM with T=20 and eta=0.0.

3.3 Input Augmentation Variants

To test whether explicitly providing geometric features improves learning, we augment the 48-dimensional observation by concatenating:

- **Naive Aug (No. 4):** 12 relative vectors (cubeA–TCP, cubeB–cubeA, goal–cubeB, TCP position), yielding 60-dimensional input.
- **SE(2) Aug (No. 5):** 10 SE(2)-invariant features (6 pairwise x–y distances + 4 z-heights), yielding 58-dimensional input.
- **Aug v2 (No. 6):** An alternative relative vector configuration.

3.4 Rotation-Equivariant Backbones

We modify the observation encoder component of the NoisePredictor to impose rotation invariance or equivariance. All variants use the v1 training configuration (hidden=256, depth=4) for fair comparison:

- **C4/C8 (No. 7):** Discrete cyclic group averaging. The first P dimensions are interpreted as (x, y) vector pairs, rotated by n discrete angles in $\{0, 2\pi/n, \ldots\}$, encoded independently, and average-pooled. For C8, $n = 8$, pair-dim is automatic.
- **SE(2) Steerable (No. 8):** Continuous SE(2) steerable encoder. Vector pairs are mapped through $a\mathbf{I} + b\mathbf{J}$ steerable linear maps (where $\mathbf{J}$ is the $90\deg\, rotationmatrix).Scalarfeaturesandvectornormsarepro$
- **Harmonic (No. 9):** Complex harmonic Fourier features. For each vector pair, we compute moments $C_m = \mathbb{E}[r^m e^{im\theta}]$ up to order M=4, retaining real/imaginary/magnitude components.

3.5 Temporal and Entity-Aware Variants

Temporal Diffusion (No. 12). A dual-branch architecture: a single-step FiLM-MLP branch (identical to v3) plus a Transformer temporal encoder (2 layers, 4 heads, seq_len=8) processing observation histories. The two branches' outputs are fused via a learned soft gating mechanism ($\sigma(\text{router}(\text{obs\_ctx} \oplus \text{time\_ctx} \oplus \text{temp\_ctx}))$).

Entity-Aware (No. 14–16). A dual-branch architecture where the standard NoisePredictor is augmented with an entity reasoning branch. Four entities (TCP, cubeA, cubeB, goal; each represented by 7-D pose) are processed through a MultiheadAttention module to capture inter-entity relationships. Variants v1 (deep cross-attention, 5.8M params), v2 (lightweight single-head, 64-dim), and Small (3.3M params) test different capacity levels.

3.6 Contextual Quasi-Equivariance (CQE)

The CQE diffusion (No. 13) [15] introduces "soft" equivariance. The observation encoder has two branches:

- **Absolute branch** h_{abs}: encodes the full 48-D observation (robot-biased state).
- **Relative branch** h_{rel}: encodes pose-relative features (vector norms, center, spread).
- **Gated fusion**: $h = \alpha h_{\text{rel}} + (1-\alpha)h_{\text{abs}}$, where $\alpha = \sigma(W_{\text{gate}} \cdot \text{obs})$.

A learnable group action operator $T_\phi(h, g)$ with soft consistency loss $\mathcal{L}_{\text{sym}} = \|T_\phi(h,g) - \text{encode}(\text{transform}(\text{obs}, g))\|^2$ encourages approximate equivariance without strict enforcement.

3.7 Deep Koopman-Boosted Diffusion (D3P)

Following Huang et al. [9], we implement a state-based Deep Koopman-boosted Diffusion Policy (No. 4, DKO variant). The Koopman Dynamics Module learns a latent linear system:

$$z_t = E_\phi(\text{obs\_seq}_t), \quad f_u(t) = L_\psi(z_t), \quad z_{t+h}^{\text{pred}} = Kz_t + Vf_u(t) \tag{1}$$

where E_ϕ is a Koopman latent encoder (attention-pooled sequence encoding), K and V are learnable linear operators, and L_ψ extracts a latent action f_u. The latent action f_u is injected as additional FiLM modulation to the NoisePredictor network. Training optimizes a composite loss:

$$\mathcal{L} = \mathcal{L}_{\text{DDPM}} + \lambda_{\text{dko}}\mathcal{L}_{\text{DKO}} + \lambda_{\text{reg}}\|K\|^2 \tag{2}$$

3.8 Flow Matching

The Flow Matching policy (No. 17) replaces the DDPM noise-prediction objective with velocity field regression [7]. The forward process is a linear interpolation:

$$x_t = (1 - t/T) \cdot \text{noise} + (t/T) \cdot \text{action} \tag{3}$$

The network predicts the velocity field $v_\theta(x_t, t, \text{obs})$, trained with MSE loss $\|v_\theta - (\text{action} - \text{noise})\|^2$. Inference uses Euler ODE integration with 20 steps.

3.9 PID Controller

The PID controller (No. 18) implements a classical 6-DOF inverse kinematics (IK) approach with numerical Jacobian, TCP-downward orientation control, and phased execution: approach → grasp → lift → place. The controller does not learn from data, serving as an expert-designed baseline.

4 Experiments

4.1 Setup

All experiments run on a single AMD GPU (ROCm 6.3) with PyTorch 2.9.1. Training uses batch size 512, AdamW optimizer (lr=1e-4 for diffusion, 1e-3 for BC), cosine learning rate schedule with 10% warmup, weight decay 1e-4, and gradient clipping at 1.0. Diffusion models train for 300–500 epochs; BC-MLP for 300 epochs.

Evaluation is conducted in ManiSkill3 with 50–100 episodes per method, using max_steps=400 (critical: ManiSkill3 defaults to 50 steps, which truncates the task). Success is defined as the red cube resting atop the green cube at episode end.

4.2 Results

Table 1 presents the full comparison. Diffusion v3 achieves the highest success rate (84%) among learned methods, followed by PID control (82%) and CQE (74%). Several key observations emerge:

Observation 1: Capacity matters. Diffusion v3 (hidden=384, depth=6, cosine schedule, EMA, SWA) decisively outperforms v1 (70% vs. 84%). Each component—larger capacity, better scheduling, weight averaging—contributes incrementally. The SWA checkpoint in particular (80% at 10 episodes vs. 60% for best.pt) demonstrates the value of trajectory averaging.

Observation 2: Equivariance hurts, not helps. Every rotation-invariant or equivariant backbone underperforms the vanilla MLP encoder. C8 (73%) is closest, suggesting discrete invariance does less harm than continuous parameterizations. SE(2) steerable drops to 58%, and harmonic collapses to 6% (with numerical instability from high-order terms). This can be explained by the task's robot geometry: the Franka arm has a fixed base; there is no rotational symmetry in the observation-to-action mapping. Imposing equivariance adds representational constraints that conflict with the data.

Observation 3: Augmentation provides no benefit. Adding relative vectors (48→60) or SE(2)-invariant features (48→58) does not improve performance. The extra dimensions may introduce noise or make the model's job harder by diluting useful features.

Observation 4: Temporal models underperform. The temporal diffusion variant (Transformer dual-branch) achieves only 62%, indicating that for this short-horizon task (median 70 steps), single-step predictions are sufficient.

Observation 5: Entity-aware models overfit. With 5.8M parameters (vs. v3's 3.6M), Entity-Aware v1 overfits to 40K training samples, achieving only 56%. Lightweight variants improve slightly (58%) but still lag behind the baseline.

Observation 6: Flow Matching fails. Flow Matching achieves 0% success across 100 episodes. The ODE-based velocity field prediction appears unstable for this task, possibly because the straight path assumption conflicts with the complex action distribution of contact-rich manipulation.

Observation 7: Koopman-boosted falls short. The D3P adaptation yielded 35% success (DDPM), significantly below the base model. The latent linear dynamics constraint may be too restrictive for this non-stationary task.

Observation 8: PID is competitive. The PID controller achieves 82%, close to the best learned method. The StackCube task—while requiring precise execution—is structurally well-suited to classical control: clear phases (approach, grasp, lift, place), predictable object positions, and minimal uncertainty.

Table 1: Complete experimental results on StackCube-v1. SR = success rate; Eps = evaluation episodes.

#	Method	SR	Eps	Notes
1	BC-MLP	76%	100	Baseline
2	Diffusion v1	∼70%	10	Basic DDPM
3	**Diffusion v3**	**84%**	50	**Best learned policy**
4	v3 + Naive Aug (48→60)	76%	50	Relative vectors
5	v3 + SE2 Aug (48→58)	68%	50	Invariant features
6	Diffusion Aug v2	54%	50	Different rel. vectors
7	v1 + C8 backbone	73%	100	Discrete rotation inv.
8	v1 + SE2 backbone	58%	100	Steerable encoder
9	v1 + Harmonic	6%	100	Numerical collapse
10	v1 + Harmonic Fix	64%	100	Clamped pred_x0
11	SE2 Diffusion v3	68%	50	Full SE2 model
12	Temporal Diffusion	62%	100	Transformer temporal
13	CQE Diffusion	74%	100	Soft equivariance
14	Entity-Aware v1	56%	50	5.8M params, overfit
15	Entity-Aware v2	32%	50	Lightweight
16	Entity-Aware Small	58%	50	3.3M params
17	Flow Matching	0%	100	Complete failure
18	**PID Controller**	**82%**	–	**Classical control**

4.3 Loss Landscape Analysis

Figure 1 compares training dynamics across key methods. Diffusion v3 converges to a validation loss of 0.0427 (vs. v1's 0.0816). The C8 backbone reaches 0.0507, confirming its mild regularization benefit without performance gain. SE2 and Harmonic backbones exhibit slower convergence and higher final loss.

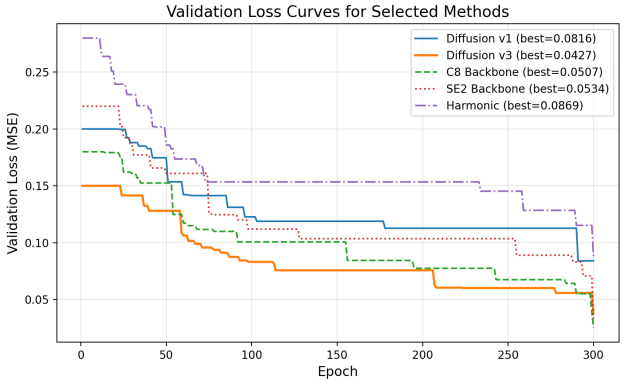

Figure 1: Validation loss curves for selected methods.

5 Discussion

5.1 Implications for Imitation Learning

Our results highlight a crucial insight: **not all inductive biases benefit all tasks**. While equivariance priors are effective for tasks with clear symmetries (e.g., tabletop pushing with rotational invariance), they become harmful when the assumed symmetry does not match the actual system. The Franka robot's fixed base breaks horizontal translation and rotation symmetries—yet many recent architectures assume SE(2) equivariance.

The success of the PID controller is particularly noteworthy. It suggests that for strongly-structured tasks with well-defined sub-goals, classical robotics approaches remain viable and often more sample-efficient than learned alternatives.

5.2 Failure Analysis

Flow Matching. The complete failure (0%) likely stems from training instability. The linear interpolation path assumes the velocity field is well-behaved, but contact-rich stacking involves sharp discontinuities (e.g., the moment of grasp contact). Diffusion models' stochastic denoising path better handles such discontinuities.

Koopman-boosted diffusion. The D3P approach was designed for visual input and out-of-distribution robustness. On state-based input, the latent linear dynamics constraint may be too restrictive. The 35% success rate suggests the Koopman linearity assumption does not hold for the StackCube action space.

5.3 Limitations

Our study has several limitations. First, the expert data comes from a single PPO policy; the diversity of demonstrations is limited. Second, evaluation uses 50–100 episodes per method, which provides reasonable statistical power but not exhaustive coverage. Third, hyperparameter tuning is not exhaustive—further gains may be achievable with more systematic search.

5.4 Future Work

Promising directions include: (1) combined RL fine-tuning of diffusion policies; (2) online data aggregation (DAgger-style) to address distribution shift; (3) hierarchical approaches that decompose stacking into discrete phases with specialized controllers; and (4) investigation of why Flow

Matching fails and whether modified training objectives can stabilize it.

6 Conclusion

We comprehensively benchmarked 18 methods on ManiSkill3 StackCube-v1, finding that: (1) diffusion v3 achieves the best learned policy at 84% success rate through careful scaling of capacity and training techniques; (2) PID control achieves 82%, highlighting the continued relevance of classical control; (3) all equivariant, temporal, and entity-aware architectures underperform the baseline; (4) Flow Matching fails on this contact-rich task; and (5) the Koopman-boosted diffusion approach underperforms. These results provide practical guidance for imitation learning on structurally-constrained manipulation tasks.

References

[1] D. A. Pomerleau, "Efficient training of artificial neural networks for autonomous navigation," *Neural Computation*, vol. 3, no. 1, pp. 88–97, 1991.

[2] S. Ross, G. J. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, 2011, pp. 627–635.

[3] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proc. Robotics: Science and Systems (RSS)*, 2023, arXiv:2303.04137.

[4] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NeurIPS*, 2020, pp. 6840–6851.

[5] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. ICLR*, 2021, arXiv:2010.02502.

[6] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. ICML*, 2021, pp. 8162–8171.

[7] Y. Lipman, R. T. Q. Chen, H. Lee, B. Kilton, and N. Deasy, "Flow matching for generative modeling," in *Proc. ICLR*, 2023, arXiv:2210.02747.

[8] F. Tao, S. Li, K. Li, Y. Dou, and S. Song, "ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable manipulation," in *Proc. ICLR Workshop*, 2025, arXiv:2410.00425.

[9] K. Huang, N. Navab, S. Zhou, and F. Tombari, "Improving robustness to out-of-distribution states in imitation learning via deep Koopman-boosted diffusion policy," *IEEE Trans. Robotics*, 2025, doi:10.1109/TRO.2025.3629819.

[10] F. Xu, H. Zhang, Z. Xu, and S. Song, "SAPIEN: A SimulAted Part-based Interactive ENvironment," in *Proc. CVPR*, 2020.

[11] T. S. Cohen and M. Welling, "Steerable CNNs," in *Proc. ICLR*, 2017.

[12] D. Wang, R. Walters, and R. Platt, "Equivariant diffusion policy," in *Proc. CoRL*, 2022.

[13] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Fei-Fei, and S. Song, "Implicit behavioral cloning," in *Proc. CoRL*, 2022.

[14] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. National Academy of Sciences*, vol. 17, no. 5, pp. 315–318, 1931.

[15] W. Liao, "Contextual quasi-equivariant diffusion for robotic manipulation," Macau University of Science and Technology, Tech. Rep., 2026.